

Documentation Catch Up

08/13/2026 15:12:52

<https://chatgpt.com/g/g-p-6a71ed6ea4d0819190118e1b9a94bea9/c/6a7dbc4f-a590-83ea-abad-86b566f24107>

e52efce5-617e-4a8a-a79c-ab65761baec9.png 0cbebc6-790e-4482-...

Thinking ● Planning documentation updates I'm treating this as one...

can i say Git Push Pray team instead of listing names?

Yes. For these meeting records, you can use: ****Attendees:**** Git Push Pr...

gitea-export-20260812-134658(1).zip client name:Terence Nkoua...

Thinking 🌐 Inspecting the Project Board Archive ● Inspected project...

gitea-export-20260812-134658(1).zip client name:Terence Nkoua...

Thinking 🌐 Inspecting Project Board ZIP Contents ● Inspected project...

1. how to do linter and prettier etc? 2. dont i need to add ai usage to cs...

Thinking 🌐 Inspecting Repository Export Contents ● Inspected reposito...

1. how to do linter and prettier etc? 2. dont i need to add ai usage to cs...

Thinking 🌐 Inspecting project archive structure ● Inspected project...



e52efce5-617e-4a8a-a79c-ab6...



0cbebce6-790e-4482-9289-9b...

we need to catch up on some documentation

1. we need initial client meeting (4 August)- discussed that he wants us to start by creating a view and log in, he is happy for us to chose our sport, we will meet every Tuesday, he gave us

[<https://www.openpowerlifting.org/>]

(<https://www.openpowerlifting.org/>) as a reference (no need to keep sport)

2. second client meeting (11 August)- demonstrated the frontend, and login capabilities. discussed that supabase limit on the free plkan is too low and therefore we had only a few data peices showing. we will work on the full db, and a login process (login, ask admin aces, show submission board if not admin) and oif we get the database sorted, some statistics

3. sprint standup on 6 August- discussed the setup we had started doing and how we plan to move forward, decided a sport and db as a group, and all took to continue what we were doing

follow project methodology etc

and what branch should i make etc?

8/13/2026, 2:51:56 PM



Thinking

- **Planning documentation updates**

I'm treating this as one documentation issue with three related meeting records. I'll suggest the issue setup, branch name, file structure, and ready-to-paste drafts, while

preserving the requested dates and repository conventions.

- **Planned documentation updates**
- **Backfilling Sprint 1 records**

I'm treating this as one documentation issue containing three related meeting records. I'll keep the dates, requested topics, and repository conventions aligned, while distinguishing the August 6 standup from your existing planning record.
- **Backfilled Sprint 1 records**
- **Created project documentation, branch guidelines, and retrospective evidence records**

✓ Worked for 1m 23s

Yes. I would handle this as **one documentation issue + one documentation branch**, because all three files serve the same purpose: catching up the missing Sprint 1 meeting evidence. Your methodology explicitly requires Tuesday stakeholder notes and Thursday standup summaries as repository evidence, and substantial documentation should follow `issue → branch → commits → PR → merge`.

`Project_Methodology_v1.1.md` `Git_Methodology.md`

Also, because you are writing these on **13 August after the meetings happened**, I would say so in the files. Keep the actual meeting date, but add `Record created: 13 August 2026 (retrospective summary)`. That is much better than making it look as though the notes were written on the day.

Your Sprint 1 rubric specifically rewards evidence of stakeholder interaction and actual use of the project methodology, so these are worthwhile records to add.

`project_briefs.pdf`

1. Create this Gitea issue first

Title:

Add missing Sprint 1 stakeholder meeting and standup records

Template: Task

Labels: `type: documentation`, `area: documentation`

Milestone: `Sprint 1`

Board: `Ready` → move to `In Progress` when you start

Description

Add the missing Sprint 1 meeting records for the stakeholder meetings held on 4 August and 11 August 2026, and the internal team standup held on 6 August 2026.

The records must follow the meeting evidence requirements defined in the project methodology and accurately capture the requirements, feedback, decisions and actions discussed.

Because the records are being added after the meetings took place, each record must clearly identify that it is a retrospective summary created on 13 August 2026.

Motivation

The project methodology requires brief records of weekly stakeholder meetings and Thursday team standups.

These records provide evidence of stakeholder interaction, project coordination, requirement clarification and the team's continued use of the agreed lightweight Scrumban methodology.

Acceptance Criteria

- [] A stakeholder meeting record for 4 August 2026 is added.
- [] A team standup record for 6 August 2026 is added.
- [] A stakeholder meeting record for 11 August 2026 is added.
- [] Each record clearly identifies that it was created retrospectively on 13 August 2026.
- [] Stakeholder requirements, feedback, decisions and resulting actions are accurately recorded.
- [] Relevant Gitea issues are linked where known.
- [] Stakeholder feedback from 11 August is linked to the submitter-access work and relevant database/statistics work.
- [] Each document contains the required AI declaration.
- [] Documentation navigation is updated where applicable.
- [] The changes are reviewed and merged through a Pull Request.

Dependencies

None.

This work records meetings and decisions that have already taken place.

Evidence

- Stakeholder meeting held 4 August 2026.
- Team standup held 6 August 2026.
- Stakeholder meeting held 11 August 2026.
- Existing Gitea issues and Project board activity resulting from these discussions.

AI Declaration

> The preceding issue was planned and generated with the assistance of ChatGPT-Web[GPT-5.6 Sol].

2. Then make this branch

Once Gitea gives you the issue number, use:

```
docs/<issue-number>-add-sprint-1-meeting-records
```

So if it becomes `#121`, for example:

```
docs/121-add-sprint-1-meeting-records
```

That exactly matches your Git methodology's required `<type>/<issue-number>-<short-description>` structure, with `docs` being the approved type for documentation-only work. `Git_Methodology.md`

```
git switch main
git pull origin main
git switch -c docs/<issue-number>-add-sprint-1-meeting-records
```

I would keep the records alongside your existing Sprint 1 evidence:

```
evidence/sprints/sprint-1/
├─ 2026-08-04-stakeholder-meeting.md
├─ 2026-08-06-planning.md
```

└─ 2026-08-06-standup.md
└─ 2026-08-11-stakeholder-meeting.md

The **6 August** **standup** is **separate from your existing Sprint 1 planning record**. Don't replace the planning record or call the standup "Sprint Planning"; your methodology treats sprint planning and the weekly Thursday standup as different evidence. `Project_Methodology_v1.1.md`

3. `2026-08-04-stakeholder-meeting.md`

Stakeholder Meeting

****Date:**** 4 August 2026

****Attendees:**** [Team members present], [Stakeholder/client name]

****Purpose:**** Initial stakeholder meeting to establish the first development priorities and clarify the direction of the Sport Analytics Tool.

****Record created:**** 13 August 2026 (retrospective summary)

Progress Demonstrated

This was the initial stakeholder meeting and focused primarily on establishing project direction and initial requirements rather than demonstrating completed functionality.

The team discussed the project brief, intended development approach and the first functionality to implement.

Requirements Discussed

The stakeholder asked the team to begin by developing:

- an initial frontend view of the application; and
- user login functionality.

The stakeholder confirmed that the team may select the sport used for the Sport Analytics Tool.

The stakeholder provided OpenPowerlifting as a reference example:

<https://www.openpowerlifting.org/>

The reference was provided as an example of an existing sport statistics platform and did not require the team to use powerlifting as its selected sport.

The team and stakeholder also agreed to meet weekly on Tuesdays for progress demonstrations, requirement clarification and feedback.

Feedback Received

The stakeholder recommended beginning with visible application functionality and authentication so that progress could be demonstrated early.

The team was given flexibility to select the sport it considered appropriate for implementing the project requirements.

OpenPowerlifting could be reviewed for ideas relating to presenting and browsing sporting data, without requiring the team to copy its sport or implementation.

Decisions

- The team will select the sport for the project.
- Initial development will include a basic frontend view.
- Authentication and login functionality will be an early priority.
- Stakeholder meetings will normally take place every Tuesday.
- OpenPowerlifting will be treated as a reference rather than a required design or sport choice.

Actions

- Finalise the team's selected sport.
- Continue initial frontend development.
- Begin authentication and login implementation.
- Review OpenPowerlifting for relevant ideas.
- Record resulting development work as Gitea issues.
- Prepare progress for the next Tuesday stakeholder meeting.

Related Gitea Issues

- [Add relevant initial frontend issue]

- [Add relevant authentication/login issue]
- [Add sport/data modelling issue if applicable]

AI Declaration

> The preceding document was generated and edited with the assistance of ChatGPT-Web[GPT-5.6 Sol].

The authentication priority is also consistent with the course-wide requirement that users be able to sign up/sign in, while your Sport Analytics brief specifically requires submission to be limited to approved submitters. `sport_analytics.pdf`

4. `2026-08-06-standup.md`

Your methodology does **not** require formal standup minutes. It asks for a short record of important progress, blockers, decisions, responsibility changes and next actions.

`Project_Methodology_v1.1.md`

Weekly Team Standup

****Date:**** 6 August 2026

****Sprint:**** Sprint 1

****Attendees:**** [Team members present]

****Record created:**** 13 August 2026 (retrospective summary)

Important Progress

The team reviewed the initial project setup work completed since the project began.

Members had started work on their respective setup and foundation tasks and discussed how these pieces would contribute to the Sprint 1 implementation.

The team also reviewed the overall direction of the project following the initial stakeholder meeting.

Blockers

No major project-wide blocker was identified at this stage.

The team recognised that the database and data foundations would be important dependencies for later submission and statistics functionality.

Decisions

- The team confirmed its selected sport.
- The team agreed on the initial database approach.
- Existing team members would continue with the work already assigned to them.
- The project would continue to prioritise the required foundations before moving into larger feature implementation.

Responsibility Changes

No major responsibility changes were made.

Team members continued with their existing assigned work.

Actions for the Following Week

- Continue current Sprint 1 setup and implementation work.
- Progress the frontend and authentication foundations.
- Continue database and data setup.
- Keep Gitea issues and the Project board updated as work progresses.
- Prepare demonstrable frontend and login progress for the next stakeholder meeting.
- Raise any new blockers through the relevant Gitea issue.

AI Declaration

> The preceding document was generated and edited with the assistance of ChatGPT-Web[GPT-5.6 Sol].

5. 2026-08-11-stakeholder-meeting.md

Stakeholder Meeting

****Date:**** 11 August 2026

****Attendees:**** [Team members present], [Stakeholder/client name]

****Purpose:**** Demonstrate Sprint 1 progress, discuss the database limitation and confirm the next priorities for authentication, submitter access and

statistics.

****Record created:**** 13 August 2026 (retrospective summary)

Progress Demonstrated

The team demonstrated the current frontend implementation and login capabilities.

A small amount of project data was also available through the application.

The team explained that the amount of data currently available was limited because the Supabase free-tier storage capacity was insufficient for the intended dataset.

Requirements Discussed

The database limitation was discussed as a priority because the application requires substantially more event data than could currently be stored using the available Supabase free-tier capacity.

The stakeholder and team discussed the required submitter-access flow.

The intended flow is:

1. A user signs in.
2. An approved submitter is given access to the submission interface.
3. A signed-in user who is not yet an approved submitter can request submitter access.
4. An administrator can review and approve access requests.
5. Once approved, the user can access the submission interface.

The team also discussed beginning statistic functionality once a suitable database solution and sufficient event data are available.

Feedback Received

The stakeholder was shown the progress made on the frontend and login functionality.

The next priorities identified were:

- resolve the database capacity problem;
- complete the submitter-access and approval flow;
- expose the submission interface only to approved submitters; and
- begin implementing statistics if the database and data foundations are ready.

Decisions

- The current Supabase free-tier capacity is insufficient for the intended project dataset and requires further action.
- Resolving the database/data limitation is a high priority.
- Submission functionality will be permission controlled.
- Users who are not approved submitters must be able to request access.
- Administrators will be responsible for approving submitter-access requests.
- Approved submitters will receive access to the submission interface.
- Statistic implementation should begin once the required database and data foundations are available.

Actions

- Investigate and resolve the database capacity limitation.
- Complete the submitter-access request and administrator approval workflow.
- Ensure the frontend displays the correct interface based on the signed-in user's approval status.
- Continue work towards a complete dataset.
- Begin initial statistic functionality when sufficient event data is available.
- Update or create the corresponding Gitea issues and priorities.

Related Gitea Issues

- #120 - Submitter access request and administrator approval flow
- [Add database capacity / replacement issue]
- [Add dataset ingestion issue]
- [Add statistics issue]

AI Declaration

> The preceding document was generated and edited with the assistance of ChatGPT-Web[GPT-5.6 Sol].

That access workflow is especially worth recording because it directly reflects the actual project requirement that **only approved submitters** may submit event data.

sport_analytics.pdf

6. Commit it

I would do **one commit** because this is one coherent documentation unit:

```
docs(evidence): add Sprint 1 meeting records
```

Commit body:

```
Refs #<issue-number>
```

You don't need to put an `Assisted-by` code footer on this commit merely because I helped write the Markdown: the course AI policy distinguishes written-document declarations from the commit attribution required for AI-generated **code**. Each Markdown document itself should carry its AI declaration, which we've included.

ai_policy_20_jul.pdf

Then open a PR such as:

```
#<issue-number> Add missing Sprint 1 meeting records
```

and use:

```
Closes #<issue-number>
```

in the PR body.

So first create that issue and send me the issue number. Then I'll give you the exact branch command, commit command, and full PR description with the right number inserted.

8/13/2026, 2:51:59 PM



can i say Git Push Pray team instead of listing names?

8/13/2026, 2:57:04 PM



Yes. For these meeting records, you can use:

****Attendees:**** Git Push Pray team, [Stakeholder/client name]

and for the internal standup:

****Attendees:**** Git Push Pray team

That is reasonable **provided the whole team attended**. If someone was absent, list the actual attendees instead so the evidence stays accurate.

I'd use the team name consistently across all three retrospective records.

8/13/2026, 2:57:07 PM



gitea-export-20260812-13465...

client name:Terence Nkoua Mackyta
ive attached our project board, give me the docs again
with everything filled in

8/13/2026, 2:58:46 PM



Thinking



Inspecting the Project Board Archive

- **Inspected project board archive and identified relevant issues and pull requests**

✓ Worked for 38s

8/13/2026, 2:58:48 PM



gitea-export-20260812-13465...

client name: Terence Nkoua Mackyta
ive attached our project board, give me the docs again
with everything filled in
also, this is issue 125

8/13/2026, 2:59:26 PM



Thinking



Inspecting Project Board ZIP Contents

- **Inspected project board exports and mapped relevant issue titles**

- **Drafting project docs**

I'm preparing issue and branch documentation for the export, preserving the Thursday standup and Tuesday stakeholder milestones. Because no exact work items are specified, I'm using related issues and clearly marking assumptions.

- **Drafted project documentation**

- **Mapping related issues**

I'm linking each meeting to the closest exported issues, while treating #120 as confirmed project context even if it isn't in the export. I'm avoiding unrelated tickets and marking assumptions where the source lacks exact issue references.

- **Mapped related issues**

- **Used personal_context tool**

✓ Worked for 2m 31s

Yes. I went through the attached Gitea export and matched the records to the issues that actually exist on your board. Since the documentation catch-up is **#125**, use:

```
git switch main  
git pull origin main
```

```
git switch -c docs/125-add-sprint-1-meeting-records
```

I would add these three files:

```
evidence/sprints/sprint-1/2026-08-04-stakeholder-meeting.md
evidence/sprints/sprint-1/2026-08-06-standup.md
evidence/sprints/sprint-1/2026-08-11-stakeholder-meeting.md
```

I've also made the retrospective nature explicit so you are not pretending the notes were written on the meeting dates.

```
:::writing{variant="document" id="58214" title="4 August 2026 Stakeholder Meeting"}
```

Stakeholder Meeting

Date: 4 August 2026

Attendees: Git Push Pray team, Terence Nkoua Mackyta

Purpose: Initial stakeholder meeting to establish the project direction, clarify the first development priorities and agree on ongoing stakeholder interaction.

Record created: 13 August 2026 (retrospective summary)

Progress Demonstrated

This was the initial stakeholder meeting, so the discussion focused primarily on establishing the direction of the Sport Analytics Tool and identifying the first areas of development rather than demonstrating completed functionality.

The team discussed the project brief and the initial approach to setting up the application.

Requirements Discussed

Terence asked the team to begin by creating:

- an initial frontend view of the application; and
- login functionality.

The team was given freedom to select the sport used for the Sport Analytics Tool.

Terence provided [OpenPowerlifting](#) as a reference example of an existing sport-statistics platform. The reference was intended to provide ideas for presenting and

exploring sporting data and did not require the team to use powerlifting or retain the same sport.

The team and stakeholder agreed to meet every Tuesday to demonstrate progress, discuss requirements and receive feedback.

Feedback Received

Terence encouraged the team to start with visible application functionality and authentication so that useful progress could be demonstrated early.

The team was given flexibility to select a sport that would work well with the event-driven analytics requirements of the project.

OpenPowerlifting was identified as a useful reference for the general concept of a public sports-statistics platform.

Decisions

- The team may select the sport used for the project.
- The first user-facing work should include an initial frontend view.
- Login and authentication should be implemented early.
- Stakeholder meetings will take place every Tuesday.
- OpenPowerlifting will be used as a reference only and does not constrain the project to powerlifting.

Actions

- Select and formally define the sport and competition scope.
- Continue development of the initial frontend.
- Establish the authentication foundation.
- Build the frontend authentication flow.
- Review OpenPowerlifting for useful ideas relating to sports-data presentation.
- Prepare demonstrable progress for the next Tuesday stakeholder meeting.

Related Gitea Issues

- #37 - Confirm the sport, professional competition scope, event vocabulary and required statistics

- #39 - Build frontend authentication pages and navigation controls
- #64 - Implement frontend authentication state, protected routes and the authenticated API client
- #89 - Add branded public landing page and application shell
- #125 - Add missing Sprint 1 stakeholder meeting and standup records

AI Declaration

The preceding document was generated and edited with the assistance of ChatGPT-Web[GPT-5.6 Sol].

...

For the **6 August standup**, I would record the actual foundational work the board shows was underway at that point: CI, Cloudflare, Supabase, OAuth and Azure, plus the team decisions around T20 cricket and the database.

:::writing{variant="document" id="74361" title="6 August 2026 Team Standup"}

Weekly Team Standup

Sprint: Sprint 1

Date: 6 August 2026

Attendees: Git Push Pray team

Record created: 13 August 2026 (retrospective summary)

Important Progress

The team reviewed the initial project setup work that had begun during the first week of Sprint 1.

Work was underway across the main project foundations, including:

- CI and reproducible project setup;
- public documentation deployment;
- Supabase/PostgreSQL database setup;
- authentication and OAuth investigation;
- Azure backend deployment;
- Azure frontend deployment; and

- general project planning and documentation.

The team discussed how these separate foundation tasks would fit together and how development should proceed once the initial setup work was complete.

The team also discussed and confirmed the initial sport and database direction.

T20 cricket was selected as the project domain, with real event-level cricket data to be sourced from Cricsheet.

Supabase-hosted PostgreSQL was selected as the initial database foundation, while the application would continue to use the team's own backend HTTP API rather than exposing Supabase-generated database endpoints directly.

Blockers

No major team-wide blocker preventing Sprint 1 work was identified during the standup.

The team recognised that successful database setup, authentication, deployment and development tooling were important dependencies for the later event-submission and statistics functionality.

Decisions

- The project will initially focus on T20 cricket.
- The project will use event-level cricket data rather than manually entered statistic totals.
- Cricsheet will be used as the primary source of representative T20 event data.
- Supabase-hosted PostgreSQL will be used as the initial database foundation.
- The frontend will not communicate directly with the Supabase database.
- The team will continue using its hand-written backend API.
- Team members will continue with the setup and implementation work already assigned to them.
- Work will continue using the agreed Git and lightweight Scrumban project methodology.
- Gitea issues and the Project board will remain the source of truth for project work.

Responsibility Changes

No major responsibility changes were made during the standup.

Each team member continued with the work they had already started or been assigned.

Actions for the Following Week

- Continue CI and reproducible development-environment work.
- Continue documentation-site deployment.
- Complete the Supabase/PostgreSQL development foundation.
- Continue authentication implementation.
- Continue Azure frontend and backend deployment work.
- Formalise the T20 cricket domain and required statistics.
- Continue architecture and Sprint 1 planning.
- Keep issue status and Project board columns updated as work progresses.
- Prepare working frontend and authentication functionality for the next stakeholder meeting.

Related Gitea Issues

- #10 - Restore green CI and reproducible installation
- #11 - Deploy the public documentation site with Cloudflare Pages
- #13 - Establish the Supabase PostgreSQL foundation
- #14 - Investigate and establish the OAuth/OIDC authentication foundation
- #15 - Establish the Azure backend deployment foundation
- #17 - Establish Azure frontend deployment foundation
- #29 - Deploy MkDocs documentation to Cloudflare Pages using Wrangler
- #36 - Create and prioritise the complete project backlog
- #37 - Confirm the sport, professional competition scope, event vocabulary and required statistics
- #38 - Produce the complete system architecture, data flow and development roadmap
- #39 - Build frontend authentication pages and navigation controls

- #125 - Add missing Sprint 1 stakeholder meeting and standup records

AI Declaration

The preceding document was generated and edited with the assistance of ChatGPT-Web[GPT-5.6 Sol].

...

And this is the important **11 August stakeholder record**, because it shows that you actually demonstrated work, identified the Supabase limitation, and turned stakeholder feedback into the next set of work.

```
:::writing{variant="document" id="91635" title="11 August 2026 Stakeholder Meeting"}
```

Stakeholder Meeting

Date: 11 August 2026

Attendees: Git Push Pray team, Terence Nkoua Mackyta

Purpose: Demonstrate current Sprint 1 progress, discuss the database limitation and agree on the next priorities for submitter access, data and statistics.

Record created: 13 August 2026 (retrospective summary)

Progress Demonstrated

The team demonstrated the current frontend implementation to Terence.

The demonstration included the application's frontend interface and working login capabilities using the Supabase authentication foundation.

The team also demonstrated the sporting data currently available through the application.

Only a limited amount of data could be demonstrated because the storage available on the current Supabase free plan was too small for the full intended T20 cricket dataset.

Requirements Discussed

The team discussed the Supabase storage limitation and the need to establish a database solution capable of supporting substantially more event-level T20 cricket data.

The required user and submitter-access flow was also discussed.

The agreed intended flow is:

1. A user visits the application and may browse public data.
2. A user can log in or sign up.
3. The application determines whether the authenticated user is an approved submitter.
4. An approved submitter is given access to the submission interface.
5. An authenticated user who is not yet an approved submitter can request submitter access.
6. An administrator can review the request and approve the user.
7. Once approved, the user gains access to the submission interface.

Submission access therefore depends on application-level approval and must not be granted merely because a user has successfully authenticated.

The team also discussed beginning implementation of event-derived statistics if the database limitation can be resolved and sufficient event data becomes available.

Feedback Received

Terence reviewed the frontend and login functionality that had been implemented since the initial stakeholder meeting.

The next priorities identified through the discussion were:

- resolve the database capacity problem;
- obtain and store sufficient T20 cricket event data;
- complete the approved-submitter access flow;
- allow non-approved authenticated users to request submitter access;
- allow administrators to approve those requests;
- expose the submission interface only to approved submitters; and
- begin implementing statistics once the required database and data foundations are available.

Decisions

- The existing Supabase free-plan capacity is insufficient for the full intended dataset.
- The team must investigate and resolve the database capacity limitation.
- Authentication and permission to submit are separate concerns.
- Successfully logging in does not automatically make a user an approved submitter.
- Approved submitters will be able to access the event-submission interface.
- Authenticated users who are not approved submitters will be able to request submitter access.
- Administrators will be able to review and approve submitter-access requests.
- Submission permissions must be controlled by the application's role and approval data.
- Event-derived statistics should be progressed once sufficient database capacity and event data are available.

Actions

- Investigate the Supabase storage limitation and determine whether the current database host remains suitable.
- Continue building the database and loading the required T20 cricket data.
- Implement the application account, role and approved-submitter model.
- Implement the submitter-access request workflow.
- Implement administrator approval of submitter requests.
- Display the submission interface only to approved submitters.
- Continue development of the event-submission functionality.
- Begin fixture and competitor statistics once sufficient event data is available.
- Keep the relevant Gitea issues and Project board statuses updated.

Related Gitea Issues

- #13 - Establish the Supabase PostgreSQL foundation

- #28 - Add Cricsheet T20 dataset downloader
- #43 - Implement the application account, role, approved-submitter and scope database schema
- #44 - Implement account synchronisation, profile API and role-based authorisation
- #45 - Build administrator submitter approval and scope management
- #52 - Implement event-derived fixture statistics and the public statistics API
- #53 - Build the approved-submitter event upload and validation-results interface
- #54 - Build public fixture and competitor statistics pages
- #62 - Implement the backend database repository and transaction foundation
- #102 - Integrate Supabase authentication with sign-in, create-account and account UI
- #105 - Database round-trip latency is 185 ms and constrains API response times
- #117 - Replace separate authentication actions with one Login or Sign up entry
- #120 - Submitter access request workflow
- #125 - Add missing Sprint 1 stakeholder meeting and standup records

AI Declaration

The preceding document was generated and edited with the assistance of ChatGPT-Web[GPT-5.6 Sol].

...

For the commit, I would use:

```
git add evidence/sprints/sprint-1/
git commit -m "docs(evidence): add missing Sprint 1 meeting records" -m "Refs #125"
git push -u origin docs/125-add-sprint-1-meeting-records
```

Then your PR title should be:

#125 Add missing Sprint 1 stakeholder meeting and standup records

and it should use `closes #125` in the PR description.

8/13/2026, 2:59:27 PM



1. how to do linter and prettier etc?
2. dont i need to add ai usage to csv and upload the transcript of this chat?
3. 3\ after that, give me commits, prs etc

8/13/2026, 3:04:31 PM



Thinking



Inspecting Repository Export Contents

- Inspected repository export contents

✓ Worked for 12s

8/13/2026, 3:04:32 PM



1. how to do linter and prettier etc?
2. dont i need to add ai usage to csv and upload the transcript of this chat?
3. 3\ after that, give me commits, prs etc

8/13/2026, 3:04:44 PM



Thinking



Inspecting project archive structure

- Inspected project archive structure



Remembering

- Reviewed COMS3011A commands, coding conventions, checks, and AI CSV process

- Running checks

I'm separating Markdown formatting from CSV validation: I'll run Prettier on the Markdown files, then use the repository's standard check command for type-checking, tests, and contract validation.

- Ran checks

- Structuring the submission

I'm planning two focused commits: one for Sprint 1 meeting records, another for AI-use documentation. I'll preserve the transcript verbatim, update the existing register, then run formatting, linting, and the full project checks.

- Structured the submission

✓ Worked for 2m 16s

Yes - there are **two extra AI-evidence steps** before you commit. And because #84 changed the repository structure, **do not add this to the old shared** `ai-usage-register.csv`. Your current convention is a per-member register.

For #125, I would do it in this order.

1. Format and check the documentation

Your branch remains:

```
git switch main
git pull origin main
git switch -c docs/125-add-sprint-1-meeting-records
```

Your methodology requires the branch to be issue-linked and the PR to pass the configured formatting, linting and other checks before merge. `Git_Methodology.md`

After you've created the three meeting files, run Prettier **only on those files** first:

```
npx prettier --write evidence/sprints/sprint-1/2026-08-04-stakeholder-
meeting.md evidence/sprints/sprint-1/2026-08-06-standup.md
evidence/sprints/sprint-1/2026-08-11-stakeholder-meeting.md
```

Then:

```
npm run format:check
npm run lint
```

And finally the full repository check:

```
npm run check
```

Your repository documentation specifically says `npm run check` is the verification command before a PR and that it covers the CI-style structure, formatting, linting, type-checking, testing and build checks. `Pasted markdown.md`

I would also run:

```
git diff --check
git status
```

Do not run Prettier over the AI transcript. The transcript is supposed to remain an unedited record.

2. Yes - add the AI register entry AND this transcript

You're right.

The course AI policy requires you to identify the tool, model and purpose of AI use, and requires AI-assisted written documents to contain an AI declaration. It also explicitly discusses retaining unedited transcripts as assessment evidence.

`ai_policy_20_jul.pdf` Your repository has gone further and established transcript/register evidence as part of its AI workflow.

Your register is now

```
evidence/ai/registers/shayna-underslak.csv
```

Not:

```
evidence/ai/ai-usage-register.csv
```

Issue #84 migrated the team to per-member registers specifically to avoid everyone editing the same CSV.

Add this as a new row. Importantly, your #84 convention requires **every field to be quoted**:

```
"2026-08-13","Shayna Underslak","ChatGPT Web","GPT-5.6 Sol","Planning; documentation; review; Git and Gitea guidance","Issue #125: reconstruct and document the 4 August stakeholder meeting, 6 August team standup and 11 August stakeholder meeting; map decisions and actions to the current Gitea project work; prepare the related AI evidence and repository workflow","Three Sprint 1 meeting evidence records; related Gitea issue mappings; AI evidence guidance; commit and Pull Request guidance","Reviewed the generated records against my recollection of the meetings and the exported Gitea project board; corrected and confirmed the meeting dates, stakeholder name, project decisions and related issue mappings before use; no application code was generated in this session","evidence/ai/transcripts/shayna-underslak/2026-08-13-issue-125-sprint-1-meeting-documentation.pdf; Issue #125"
```

Make sure there is a **newline after the row**.

Transcript

I would save this conversation as:

```
evidence/ai/transcripts/shayna-underslak/2026-08-13-issue-125-sprint-1-meeting-  
documentation.pdf
```

PDF is actually convenient here because it preserves the conversation as-is and Prettier won't try to reformat it. Your existing AI evidence process already has PDF transcript examples.

Keep it **unedited**. Don't clean up my answers, remove your corrections, or make the conversation look neater. The fact that you challenged/corrected things in the conversation is useful AI evidence - the Lab 1 rubric specifically values visible constraints, corrections and redirection rather than unquestioningly accepting generated output. `lab 1.pdf`

Also check the PDF before committing it for anything you wouldn't want in the repository: credentials, tokens, private URLs, etc.

You **do not need an** `Assisted-by:` **footer on these commits**, because no AI-generated application code is being committed. The AI policy distinguishes written-document declarations from AI-generated-code commit attribution. Your three meeting documents already contain the required written AI declaration. `ai_policy_20_jul.pdf`

3. Then make two commits

I prefer **two commits**, rather than putting everything into one, because your Git methodology asks for small logical commits. `Git_Methodology.md`

Commit 1 - meeting evidence

First check exactly what you're adding:

```
git status
```

Then:

```
git add evidence/sprints/sprint-1/2026-08-04-stakeholder-meeting.md  
git add evidence/sprints/sprint-1/2026-08-06-standup.md  
git add evidence/sprints/sprint-1/2026-08-11-stakeholder-meeting.md
```

Commit:

```
git commit -m "docs(evidence): add Sprint 1 meeting records" -m "Refs #125"
```

Commit 2 - AI evidence

```
git add evidence/ai/registers/shayna-underslak.csv
git add evidence/ai/transcripts/shayna-underslak/2026-08-13-issue-125-sprint-1-meeting-documentation.pdf
```

Then:

```
git commit -m "docs(ai): record AI usage for issue 125" -m "Refs #125"
```

Now run the final check **after both commits' content is present**:

```
npm run check
git status
```

You want:

```
nothing to commit, working tree clean
```

Then make sure your branch is current with `main` before opening the PR, as required by your Git methodology. `Git_Methodology.md`

```
git fetch origin
git merge origin/main
npm run check
```

If that's all green:

```
git push -u origin docs/125-add-sprint-1-meeting-records
```

4. Pull Request

PR title:

```
#125 Add missing Sprint 1 stakeholder meeting and standup records
```

:::writing{variant="standard" id="58321"}

Purpose

Add the missing Sprint 1 stakeholder meeting and internal standup records required by the project methodology.

The records document the stakeholder meetings held on 4 August and 11 August 2026 and the internal team standup held on 6 August 2026.

Because the records were added after the meetings occurred, each document clearly identifies itself as a retrospective summary created on 13 August 2026.

Changes

This Pull Request adds:

- `evidence/sprints/sprint-1/2026-08-04-stakeholder-meeting.md`
- `evidence/sprints/sprint-1/2026-08-06-standup.md`
- `evidence/sprints/sprint-1/2026-08-11-stakeholder-meeting.md`

The records capture:

- the initial stakeholder priorities of a frontend view and login functionality;
- the team's freedom to select the project sport;
- OpenPowerlifting as a reference platform;
- the agreed weekly Tuesday stakeholder meetings;
- the team's 6 August Sprint 1 setup progress and technical direction;
- the selection of T20 cricket and the initial database approach;
- the frontend and authentication functionality demonstrated on 11 August;
- the Supabase free-plan database-capacity limitation;
- the requested submitter-access and administrator-approval workflow;
- the database and data work required next; and
- the intention to begin event-derived statistics once sufficient data is available.

Relevant existing Gitea issues are linked from the records so that stakeholder feedback and team decisions remain traceable to project work.

This Pull Request also adds the associated AI usage evidence to Shayna Unterslak's individual AI register and transcript directory.

Testing

The documentation was manually reviewed against the team's recollection of the meetings and the current Gitea project records.

The following repository checks were run:

```
npx prettier --write <meeting record files>
npm run format:check
npm run lint
npm run check
git diff --check
```

All applicable checks pass.

Known Limitations

The three meeting records are retrospective summaries rather than contemporaneous or verbatim meeting transcripts.

This is stated explicitly in each record.

No application functionality is changed by this Pull Request.

AI Usage

The meeting documentation was generated and edited with the assistance of ChatGPT-Web[GPT-5.6 Sol].

The associated usage is recorded in:

```
evidence/ai/registers/shayna-unterslak.csv
```

and the unedited session transcript is retained under:

```
evidence/ai/transcripts/shayna-unterslak/
```

No application code was generated or modified as part of this work.

Related Issue

Closes #125

:::

Your methodology explicitly requires Tuesday stakeholder notes and Thursday standup summaries as evidence, so this PR directly fills a methodology-evidence gap rather than adding documentation just for the sake of it. `Project_Methodology_v1.1.md`

5. Gitea board flow

When you're working on it:

```
#125: In Progress
```

Once the PR is open:

```
#125: In Review
```

Then have **another team member review it**. Your methodology requires peer review before merge. `Git_Methodology.md`

Once they have reviewed it, all checks pass and you've dealt with any comments:

1. Merge the PR using a **merge commit**.
2. `Closes #125` will close the issue.
3. Delete `docs/125-add-sprint-1-meeting-records`.
4. Move #125 to **Done**.
5. Pull the updated `main` locally.

```
git switch main
git pull origin main
git branch -d docs/125-add-sprint-1-meeting-records
```

So your final #125 evidence chain becomes:

```
Issue #125
  ↓
docs/125-add-sprint-1-meeting-records
  ↓
docs(evidence): add Sprint 1 meeting records
docs(ai): record AI usage for issue 125
  ↓
PR: #125 Add missing Sprint 1 stakeholder meeting and standup records
```

↓
peer review + checks
↓
merge
↓
#125 Done

That's exactly the `issue → branch → commits → Pull Request → merge` traceability your Git methodology asks you to demonstrate. `Git_Methodology.md`

8/13/2026, 3:04:45 PM